IMT Atlantique

Projet Procom

Argus

Preuve de concept et limites

SCHOBERT Néo
EL AKRABA Othmane
AYOUBI Houcine
TRAORE Idrissa

Argus – PoC & Limites

Table des matières

I Protocole de benchmarking 2

II Résultats complets 3

III Analyse des résultats 4

IV Limites identifiées 4

V Pistes d'amélioration 5

VI Conclusion 6

I Protocole de benchmarking

Rappel du workflow de benchmarking

Le workflow de benchmarking d'Argus évalue la qualité d'un modèle on-premise en le faisant noter par des juges cloud. Le modèle testé analyse le projet C++ localement, puis les juges reçoivent le code et les rapports pour évaluer leur qualité en 3 phases : analyse indépendante, révision croisée, et consensus (médiane pondérée).

Échelle de scores et critères d'évaluation

Score de vulnérabilité (0–10). Chaque nœud du graphe d'appels reçoit un score global entre 0 et 10, correspondant à un verdict d'exploitabilité :

Verdict	Plage obligatoire	Conditions
EXPLOITABLE	7.0 – 10.0	Entrée attaquant confirmée atteignant le sink, aucun garde efficace
LIKELY_EXPLOITABLE	5.0 – 8.0	Probable, comportement par défaut vulnérable, ou aucune mitigation visible
CONDITIONALLY_EXPLOIT.	3.0 – 6.0	Nécessite des options de build ou conditions d'exécution spécifiques
INSUFFICIENT_INFO	3.0 – 5.0	Impossible de trancher — inconnu $\neq$ sûr, jamais 0.0
NOT_EXPLOITABLE	0.0 – 2.0	Uniquement si une mitigation explicite, efficace et non contournable est présente

Table 1 – Verdicts et plages de scores obligatoires

Métriques qualité (0.0–1.0 par critère). Chaque juge évalue également la qualité du rapport produit par le modèle testé selon 12 critères locaux (par nœud) et 10 critères globaux. Les principaux critères locaux sont :

- **taint_analysis_quality** : la chaîne source $\rightarrow$ sink est-elle tracée avec les noms de variables ?
- **cve_version_matching** : la version affectée est-elle explicitement vérifiée ?
- **exploit_completeness** : toutes les CVEs exploitables ont-elles été détectées ?
- **speculation_control** : le rapport évite-t-il les chemins d'attaque inventés ?
- **technical_clarity** : des numéros de ligne et noms de variables sont-ils cités ?

Les critères globaux ajoutent : **exploit_chain_detection** (chaîne A $\rightarrow$ B détectée via la topologie du graphe), **critical_node_identification** (classement des nœuds les plus dangereux), et **call_graph_exploitation** (utilisation effective de la structure d'appels).

Consensus (phase 3). La médiane pondérée des scores révisés (phase 2) détermine le score final. Le poids de chaque juge est $1.0 + 0.5 \times \min(n_{\text{consensus}}, 4)$, où $n_{\text{consensus}}$ est le nombre de constats partagés avec les pairs. Le score global ne peut dépasser le maximum des scores locaux que si une chaîne d'exploitation est confirmée par au moins deux juges.

Projets de test

- **MyProject** : projet C++ minimal utilisant la bibliothèque `cpr` (wrapper C++ de libcurl) et libcurl 7.18.0 directement. Contient des appels à `cpr::Get` ainsi qu'aux fonctions de l'API curl bas niveau.
- **VulnerableCurlExample** : projet utilisant libcurl 7.18.0 (version délibérément ancienne), avec 9 points d'appel distincts. Les deux CVEs associées sont CVE-2009-0037 (suivi de redirections vers des ressources locales, CVSS 6.8) et CVE-2009-2417 (null byte injection dans les certificats SSL, CVSS 6.8).

Modèles et juges

Trois modèles on-premise ont été évalués : `devstral:24b` (Mistral AI, spécialisé code), `gpt-oss:20b` et `qwen3:32b`.

Deux juges cloud ont servi de référence : **DeepSeek Chat** et **Gemini 2.5 Flash**.

II Résultats complets

Au total, **12 tests** ont été réalisés et enregistrés en base de données.

Modèle	Projet	Score	DeepSeek	Gemini	Consensus
`devstral:24b`	MyProject (vulnérable)	8.5	7.0	7.0	7.0
	VulnerableCurlExample (run 1)	7.0	8.0	8.0	8.0
	VulnerableCurlExample (run 2)	7.0	8.5	8.5	8.5
	MyProject (non vulnérable)	0.0	0.0	0.0	0.0
`gpt-oss:20b`	MyProject (vulnérable)	5.5	9.0	9.0	9.0
	VulnerableCurlExample (run 1)	6.0	8.5	8.5	8.5
	VulnerableCurlExample (run 2)	8.0	9.0	9.0	9.0
	MyProject (non vulnérable)	0.0	0.0	0.0	0.0
`qwen3:32b`	MyProject (vulnérable)	7.5	7.5	8.5	8.0
	VulnerableCurlExample (run 1)	7.5	9.0	9.0	9.0
	VulnerableCurlExample (run 2)	7.5	8.0	9.0	8.5
	MyProject (non vulnérable)	0.0	0.0	0.0	0.0

Table 2 – Résultats des 12 tests. *Score* = score global du modèle testé. *DeepSeek* et *Gemini* = scores révisés après révision croisée. *MyProject (non vulnérable)* : version sans CVE — score 0.0 attendu et correct.

Synthèse — tests valides uniquement

En excluant les 3 tests à score nul (voir §III), voici les moyennes sur les 9 tests exploitables :

Modèle	Score moyen (testé)	Score consensus moyen	Correction
`devstral:24b`	7.5	7.8	+0.3
`gpt-oss:20b`	6.5	8.8	+2.3
`qwen3:32b`	7.5	8.5	+1.0

Table 3 – Synthèse sur VulnerableCurlExample + MyProject (tests valides). La correction représente l'écart entre le score du modèle testé et le consensus des juges.

Le mécanisme de consensus corrige systématiquement les sous-évaluations, avec une correction particulièrement marquée pour `gpt-oss:20b` (+2.3 points).

Sur **VulnerableCurlExample** uniquement (projet avec CVEs certaines) :

- `devstral:24b` : 7.0 / 7.0 → consensus 8.0 / 8.5 (correction +1.25 en moyenne)
- `gpt-oss:20b` : 6.0 / 8.0 → consensus 8.5 / 9.0 (correction +1.75 en moyenne)
- `qwen3:32b` : 7.5 / 7.5 → consensus 9.0 / 8.5 (correction +1.25 en moyenne)

III Analyse des résultats

Scores nuls : projet sans CVE

Les trois tests à score 0.0 (un par modèle) correspondent à une version de MyProject **sans CVE connue** — aucune bibliothèque avec une vulnérabilité référencée n'est utilisée dans cette configuration. Le score 0.0 est donc le résultat correct et attendu : il n'y a rien à détecter. Juges et modèle testé s'accordent unanimement sur ce verdict, ce qui illustre également la bonne calibration du système sur les projets sains.

Variance inter-runs

Sur VulnerableCurlExample, `gpt-oss:20b` présente une variance significative entre ses deux runs (6.0 vs 8.0), indiquant une instabilité du modèle sur ce type de tâche. `devstral:24b` et `qwen3:32b` sont plus stables (7.0/7.0 et 7.5/7.5).

IV Limites identifiées

- **Analyse par points d'appel** : Argus ne voit que les appels visibles dans le code. Une CVE déclenchée par la simple liaison avec une bibliothèque, via un callback implicite, ou dans le code d'initialisation statique ne sera pas détectée.
- **BackSlicer heuristique** : les templates C++ complexes, macros préprocesseur et appels via pointeurs de fonctions peuvent ne pas être correctement suivis.

- **CMake uniquement** : les projets sans CMakeLists.txt (Makefile, Bazel, Meson...) ne sont pas supportés.
- **Analyse probabiliste** : deux exécutions identiques peuvent produire des scores différents. La variance observée est de 0 point pour devstral et qwen3 sur VulnerableCurlExample, mais de 2 points pour gpt-oss.
- **Corpus limité** : 12 tests sur 2 projets ne suffisent pas pour des conclusions statistiquement robustes. Un corpus plus large (projets diversifiés, CVEs de différentes natures, code non vulnérable) serait nécessaire.
- **Tests séquentiels** : l'état partagé dans les `ClassVar` de `CallGraphNode` empêche l'exécution parallèle des tests de benchmarking.

V Pistes d'amélioration

Améliorer le BackSlicer

Le BackSlicer actuel repose sur des heuristiques regex pour remonter les flux de données. Cette approche atteint ses limites sur le C++ moderne : templates, lambdas, pointeurs de fonctions, et macros complexes ne sont pas toujours correctement suivis.

La solution idéale serait un **slicer basé sur l'AST Clang** (via `libclang` ou l'API C++ de Clang), qui produirait un vrai graphe de dépendances de données (PDG/SDG) permettant de suivre les flux avec une précision structurelle plutôt que textuelle. Cette approche n'a pas été retenue car elle impose de fournir l'ensemble des en-têtes système et des dépendances au moment de la compilation — une contrainte lourde à automatiser dans un outil générique. Le slicer heuristique actuel constitue un compromis pragmatique : il ne nécessite aucune configuration de l'environnement de compilation, au prix d'une couverture incomplète sur le C++ avancé.

Améliorer les prompts et le contexte fourni aux LLMs

Les modèles on-premise de taille modeste (24B paramètres) bénéficient fortement d'un contexte riche et structuré. Plusieurs axes d'amélioration sont envisageables :

- **Fournir le pattern canonique de la CVE** : inclure dans le prompt un exemple de code vulnérable et un exemple de code corrigé, extraits de la base CWE. Le modèle peut alors comparer directement le code analysé au pattern de référence.
- **Injecter les exploits publics (ExploitDB)** : pour les CVEs avec un exploit référencé, fournir un extrait de l'exploit comme contexte. Le modèle comprend mieux le vecteur d'attaque concret.
- **Fournir la topologie du graphe d'appels** dans le prompt global, pour permettre la détection de chaînes d'exploitation (vulnérabilité A qui facilite l'exploitation de vulnérabilité B).
- **Enrichissement CWE via cloud** : avant l'analyse on-premise, appeler un LLM cloud pour classifier la CVE par CWE et générer le contexte sémantique. Le code source ne

transitant pas dans cet appel, cela reste compatible avec les contraintes de confidentialité.

Élargir le corpus de test

12 tests sur 2 projets est insuffisant pour des conclusions robustes. Un corpus plus représentatif inclurait :

- Des projets avec des bibliothèques diversifiées (OpenSSL, zlib, libpng, SQLite...)
- Des CVEs de différentes natures (buffer overflow, integer overflow, use-after-free, format string...)
- Des projets avec des mitigations partielles (certains appels sécurisés, d'autres non)
- Des projets sans CVE comme base de mesure du taux de faux positifs

Autres améliorations

- Support de systèmes de build alternatifs (Makefile, Bazel, Meson) pour élargir la couverture
- Refactoring de `CallGraphNode` pour permettre l'exécution parallèle des tests
- Calibration fine des plages de scores par type de CVE (les vulnérabilités de configuration ont un profil d'exploitabilité différent des buffer overflows)

VI Conclusion

Argus démontre la faisabilité d'une approche hybride statique + IA pour qualifier l'exploitabilité des CVEs, avec une contrainte forte de confidentialité du code source grâce au workflow on-premise.

devstral :24b est le modèle le plus stable des trois testés (variance nulle sur VulnerableCurlExample, scores alignés avec le consensus des juges). **gpt-oss :20b** présente la correction la plus forte par le consensus (+2.3 points), signe d'une sous-évaluation systématique. **qwen3 :32b** est intermédiaire.

Le mécanisme de consensus en 3 phases apporte une valeur mesurable (+1.25 à +1.75 points sur les projets vulnérables) et la détection correcte des projets sains (score 0.0 unanime) valide la calibration basse du système.